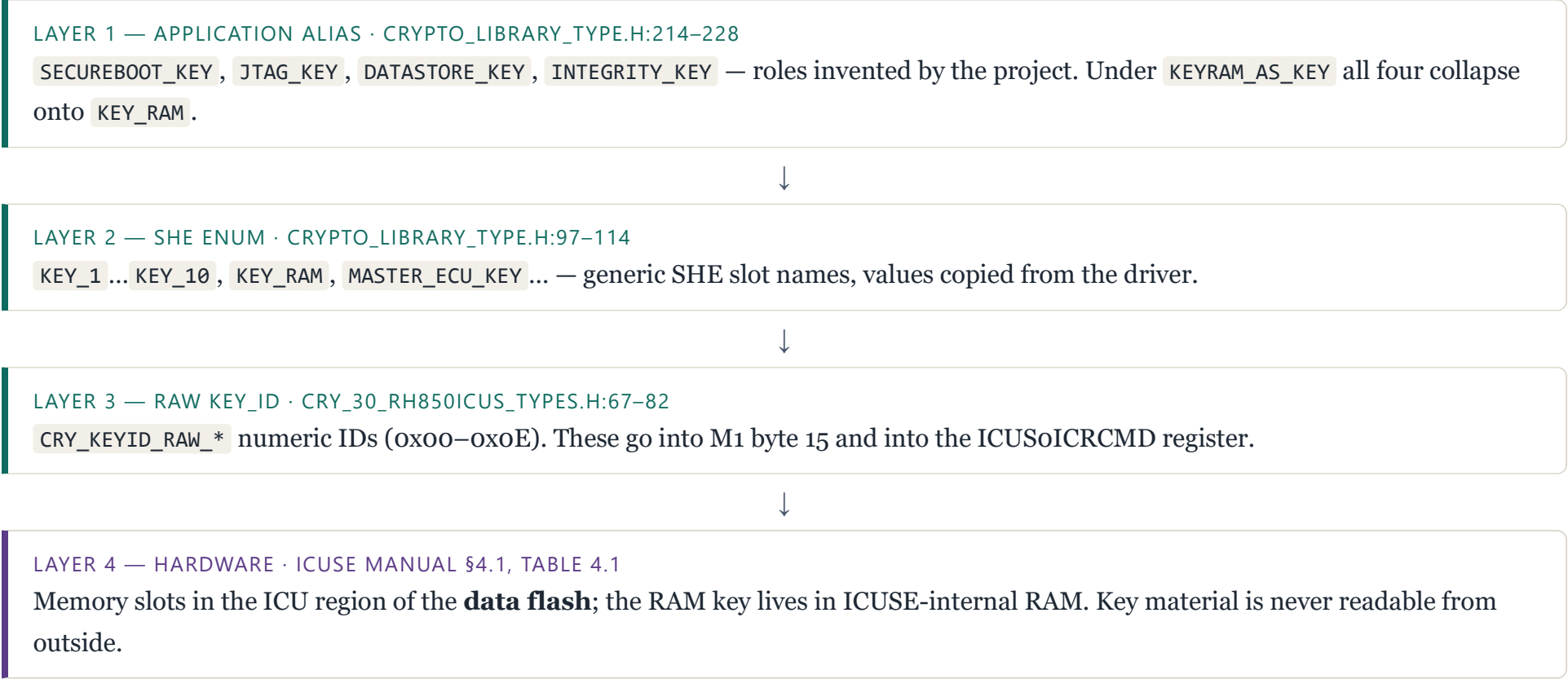


Key Slot Map & Glossary

The compressed essence of Lesson 0001 — keep this beside you when reading the security stack or the DaVinci configuration.

1 • The four name layers

A single key travels under four different names. Everything else follows from this stack:



2 • Project key assignments (default build)

APPLICATION ALIAS	SLOT	RAW KEY_ID	FLAGS AT LOAD	USED BY
SECUREBOOT_KEY	KEY_1	0x04	KU WP	CMAC over app/flasher images (SecSB_WriteImageCMAC / VerifyImageCMAC)
JTAG_KEY	KEY_2	0x05	— (loaded elsewhere)	JTAG password encryption
DATASTORE_KEY	KEY_3	0x06	KU WP	HTA / datastore protection (SecSB_EnableSecurity step 2)
INTEGRITY_KEY	KEY_4	0x07	KU WP	Integrity module (SecSB_EnableSecurity step 3)
(working key)	KEY_RAM	0x0E	none (RAM key has no flags)	Carrier for fresh TRNG material during provisioning

Anchors: secure_boot.c:145,162 (DATASTORE/INTEGRITY), secure_boot.c:361 (SECUREBOOT in keyData), crypto_library_type.h:224–227.

COMPILE-VARIANT SWITCH

With KEYRAM_AS_KEY defined (crypto_library_type.h:214–219) all four aliases become KEY_RAM : keys are then loaded as plain keys into the volatile RAM slot and do not survive power-off. The default build uses the persistent NVM slots KEY_1–KEY_4 via the full M1M2M3 protocol.

3 • SHE key ID map — ICUSE Table 4.1 vs. driver header

KEY	KEYMD	KEY_ID	DRIVER MACRO (CRY_30_RH850ICUS_TYPES.H)
SECRET_KEY (device-unique ROM key)	0	0x00	CRY_KEYID_RAW_SECRET_KEY
MASTER_ECU_KEY	0	0x01	CRY_KEYID_RAW_MASTER_ECU_KEY
BOOT_MAC_KEY	0	0x02	CRY_KEYID_RAW_BOOT_MAC_KEY
BOOT_MAC	0	0x03	CRY_KEYID_RAW_BOOT_MAC
KEY_1 ... KEY_10	0	0x04 ... 0x0D	CRY_KEYID_RAW_KEY_1 ... _KEY_10
RAM_KEY	0	0x0E	CRY_KEYID_RAW_KEY_RAM
KEY_11 ... KEY_20 (extended)	1	0x04 ... 0x0D again	CRY_KEYID_RAW_KEY_11 ... _KEY_20

KEYMD is bit 20 of the ICUSoICRCMD register — it selects the extended key page (KEY_11–20 reuse IDs 0x04–0x0D). The Vector driver masks it as CRY_30_RH850ICUS_CMD_KEYMD_MASK (Cry_30_Rh850Icus_Hw.h:109). [Manual §4.2, Table 4.1, p. 40]

4 · KEY_FLAG bits (manual Table 4.2 = code defines)

BIT	CODE (CRYPTO_LIBRARY_TYPE.H:67–76)	MANUAL NAME	EFFECT
0x80	WP	WRITE_PROTECTION	Key can never be rewritten or erased — one-way latch, cannot be cleared.
0x40	BP	BOOT_PROTECTION	Key unusable unless secure boot passed (BOOT_OK/BOOT_FINISHED in ICUSoICRSTS).
0x20	DBGP	DEBUGGER_PROTECTION (DP)	Key unusable while a debugger is attached (EXT_DEBUGGER status bit).
0x10	KU	KEY_USAGE	1 = CMAC generate/verify only; 0 = AES encrypt/decrypt only.
0x08	DWC	DISABLE_WILDCARD	1 = wildcard UID (all-zero) no longer accepted for key update.
0x04	VO	CMAC_USAGE (CU)	1 = may only <i>verify</i> CMACs, not generate them.

All six project keys are loaded with KU | WP (secure_boot.c:365,442): MAC-only use, frozen after first provisioning.

5 · The key update message M1–M5 (manual §4.4, Figure 4.1)

M1 (128 b) = UID(120) | ID(4) | AuthID(4)

M2 (256 b) = ENC_CBC,K1,IV=0 (Counter(28) | KEY_FLAG(6) | "0..."94 | KEYID(128))

M3 (128 b) = CMAC_K2 (M1 | M2)

M4 (256 b) = UID | ID | AuthID | ENC_ECB,K3 (Counter | 1 | "0..."99)

M5 (128 b) = CMAC_K4 (M4)

K1 = KDF(KEY_AuthID, KEY_UPDATE_ENC_C) K3 = KDF(KEYID, KEY_UPDATE_ENC_C)

K2 = KDF(KEY_AuthID, KEY_UPDATE_MAC_C) K4 = KDF(KEYID, KEY_UPDATE_MAC_C)

KEY_UPDATE_ENC_C = 01015348 45008000 00000000 000000B0h

KEY_UPDATE_MAC_C = 01025348 45008000 00000000 000000B0h

(extended keys KEY_11–20 use 0181.../0182... constants)

- **M1–M3 carry the registration; M4–M5 are the ICUSE's read-back proof** that the key landed correctly.
- **Self-authenticated first load:** when AuthID == ID and the auth key slot is unregistered, KEY_AuthID is treated as 0...0₁₂₈ — that is how this project provisions empty slots (secure_boot.c passes authKeyId = keyId , authKey = 0). [Manual §4.4, p. 43]
- **Counter** is a monotonic 28-bit anti-replay value; must be ≥ 1 on first registration (project uses 1), 0 only for RAM_KEY.
- **UID wildcard** = 120 zero bits; allowed only while the target key's DWC flag is 0.

6 • Code path quick map

STEP	FUNCTION / FILE	ROLE
1	SecLib_RandomGenerate · crypto_library.c	TRNG generates the 16-byte key on-target
2	SecLib_KeyRAMUpdate · crypto_library.c:397	Stages plaintext into RAM_KEY via Cry_30_Rh850Icus_KeyExtract (64-byte plain form → CMD_LOAD_PLAIN_KEY)
3	SecLib_GenerateM1M2M3 · key_manager.c:99	Builds M1 M2 M3 with KDF keys K1–K3
4	SecLib_KeySHEUpdate · crypto_library.c:456	Passes M1M2M3 to driver KeyExtract (65-byte form), keyId in symKey.data[0]
5	Cry_30_Rh850Icus_KeyExtract.c (KeyUpdate=64, WithKeyID=65)	Parses ID/AuthID from M1 byte 15, derives K4 from KEYID, returns M4M5
6	Cry_30_Rh850Icus_Hw.c:1100	SendCommand(CMD_LOAD_KEY, keyId, keyIdType) — writes the memory slot
7	SecSB_CreateKey tail · secure_boot.c:463–480	MAC generate + verify round-trip proves the slot is live

7 • Error signals worth memorising

SIGNAL	MEANING
ERC_KEY_EMPTY (bit 0x008 of ICUS error reg)	Slot was never registered — the exact state SecSB_VerifySecureBoot reports as E_VC_SECB00T_ICUS_KEY_EMPTY before provisioning.
ERC_KEY_WRITE_PROTECTED (ERR5)	Slot has WP set — re-provisioning attempt rejected. The enable path treats this as "already done", not as failure (secure_boot.c:142).
ERC_KEY_UPDATE_ERROR (ERR6)	M1M2M3 rejected: CMAC failed, counter not monotonic, UID mismatch, or flags violated.
ERC_KEY_NOT_AVAILABLE / ERC_MEMORY_FAILURE	Referenced slot does not exist / ICUSE memory self-test failed.

Code mapping: SecLib_CheckICUSError() → E_VC_SECB00T_ICUS_* enums, crypto_library_type.h:166–176. [Manual §5.10, ICUSoICRSTS/ERR register sections]

8 • Glossary

ICUSE	ICU Security unit — the RH850 on-chip SHE-class security module performing AES128/CMAC/TRNG and secure boot.
ICU region	Part of the data flash reserved by ICUSE for key slots and their flags/counters; externally unreadable.
Memory slot	One key storage area in the ICU region (NVM keys) — addressed by KEY_ID.
KEY_ID	4-bit selector of a key slot (Table 4.1); appears in M1, in ICUSoICRCMD[19:16], and in driver macros.
KEYMD	Bit 20 of ICUSoICRCMD; selects the extended key page (KEY_11–20).
KEY_FLAG	6-bit attribute set stored with the key at registration: WP, BP, DP, KU, DWC, CU/VO.
AuthID	KEY_ID of the key that authenticates the update of another key; equals ID for self-authenticated first loads.
UID	120-bit device identifier read via CMD_GET_ID; all-zero value = wildcard (if DWC=0).
Counter (CID)	28-bit monotonic anti-replay counter stored with the key; updates require a strictly greater value.
M1–M5	SHE key-update messages: M1 (UID ID AuthID), M2 (encrypted key+flags+counter), M3 (authenticating CMAC), M4/M5 (ICUSE's verification proof).
K1–K4	

Session keys derived via SHE KDF from the auth key (K1,K2) and the new key (K3,K4) with the KEY_UPDATE_*_C constants.

CMD_LOAD_KEY

ICUSE command 0008h — register/update an NVM key (or RAM_KEY) from M1–M3.

CMD_LOAD_PLAIN_KEY

ICUSE command registering a plaintext key into RAM_KEY (no M1M2M3 protection).

CMD_EXPORT_RAM_KEY

Exports RAM_KEY encrypted under SECRET_KEY for later re-import via CMD_LOAD_KEY.

RAM_KEY

Single volatile key in ICUSE-internal RAM; no KEY_FLAG, no counter; survives only until reset unless exported.

TRNG

True random number generator inside ICUSE; source of all key material in this project.

WP / one-way latch

Once WRITE_PROTECTION is set at registration, the slot is immutable until the next full chip erase.

Sources: RH850/F1KM ICUSE User Manual R01UH0703EJ0100 Rev.1.00 (§4.1–4.4, §5.6.7, §5.10) · Vector Cry_30_Rh850Icus / Crypto_30_CryWrapper implementation · project `security/` stack.

Companion lesson: 0001 — Where do SECUREBOOT_KEY and DATASTORE_KEY actually live?